

PANDUAN IMPLEMENTASI

Otomatisasi Detector Timeout Arsitektur Vercel & Restoring VPS

Dokumen ini berisi panduan lengkap langkah demi langkah untuk membangun sistem pemantauan otomatis (AI Watcher) pada arsitektur web yang menggunakan Vercel sebagai frontend dan VPS sebagai backend API. Ketika terjadi masalah `ERR_TIMED_OUT` atau `504 Gateway Timeout`, sistem akan otomatis melakukan perbaikan jarak jauh melalui protokol SSH aman.

Langkah 1: Konfigurasi Skill Agen Antigravity

Langkah pertama adalah mendaftarkan kapabilitas atau keahlian baru ke dalam lingkungan agen kecerdasan buatan (AI Agent) Anda di dalam project agar dapat mengenali perintah pemantauan secara otomatis.

Buat sebuah folder bernama `.agents/skills/vercel-watcher/` di dalam direktori utama (root) proyek Anda, kemudian buat sebuah file baru bernama `SKILL.md` di dalamnya dan isi dengan baris kode berikut:

```
---
id: vercel-timeout-watcher
description: Mendeteksi timeout pada domain utama (Vercel) dan otomatis merestore service backend jika terdeteksi overload.
---

## Kriteria Aktivasi
Gunakan skill ini ketika pengguna meminta untuk menjalankan pemantauan otomatis pada arsitektur Vercel dan backend VPS, atau ketika ekosistem mendeteksi kegagalan respon jaringan secara berulang.

## Langkah Prosedural
1. Jalankan skrip `watcher.py` untuk mulai mendeteksi HTTP response.
2. Jika skrip mendeteksi kegagalan koneksi (Timeout), jalankan perintah SSH khusus.
3. Kirimkan log pemulihan (restoring status) ke panel Antigravity secara real-time.
```

Langkah 2: Pembuatan Skrip Detektor Utama (watcher.py)

Skrip Python ini berfungsi sebagai mesin pemantau aktif yang berjalan di latar belakang. Skrip akan mengirimkan permintaan (ping/request) secara berkala ke domain Vercel Anda dan mengeksekusi skrip pembersihan internal VPS jika terdeteksi pembekuan server.

Buatlah berkas bernama `watcher.py` di root direktori proyek Anda, lalu masukkan seluruh kode terstruktur berikut:

```
import time
import requests
```

```

import paramiko

# --- KONFIGURASI INTEGRASI ---
TARGET_URL = "https://sapatamu.id" # Alamat domain Vercel Anda
SSH_HOST = "123.45.67.89" # Ganti dengan IP Address VPS Anda
SSH_USER = "root" # Ganti dengan username SSH Anda
SSH_PASSWORD = "PasswordVPSAnda" # Masukkan password VPS Anda

def restart_backend_service():
    print("[🔥 ALERT] Terdeteksi Timeout! Menghubungi VPS untuk proses restorasi...")
    try:
        ssh = paramiko.SSHClient()
        ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())

        # Membuka koneksi SSH aman menuju VPS
        ssh.connect(SSH_HOST, username=SSH_USER, password=SSH_PASSWORD)

        # Perintah kombinasi untuk membersihkan memory cache, restart PM2, dan restart Nginx
        cmd = "sudo sync && echo 3 | sudo tee /proc/sys/vm/drop_caches && pm2 restart all && sudo systemctl restart nginx"
        stdin, stdout, stderr = ssh.exec_command(cmd)

        print("[✅ SUCCESS] Perintah restorasi terkirim. Cache RAM dibersihkan & service di-restart.")
        print(stdout.read().decode())
        ssh.close()
    except Exception as e:
        print(f"[❌ ERROR] Gagal melakukan eksekusi SSH untuk restorasi: {e}")

def monitor_site():
    print(f"[🔍 START] Memulai proses AI Watcher otomatis untuk {TARGET_URL}...")
    while True:
        try:
            # Toleransi batas waktu tunggu (timeout) diset 10 detik
            response = requests.get(TARGET_URL, timeout=10)

            if response.status_code == 200:
                print(f"[{time.strftime('%H:%M:%S')}] Status Respon: OK (200)")
            elif response.status_code in [502, 504]:
                print(f"[{time.strftime('%H:%M:%S')}] Server Error terdeteksi ({response.status_code})!")
                restart_backend_service()
                time.sleep(60) # Memberikan jeda 1 menit setelah merestart agar tidak looping
            else:
                print(f"[{time.strftime('%H:%M:%S')}] Status Respon: {response.status_code}")

        except requests.exceptions.Timeout:
            print(f"[{time.strftime('%H:%M:%S')}] Kritis! Terdeteksi ERR_TIMED_OUT!")
            restart_backend_service()
            time.sleep(60)

```

```
except requests.exceptions.RequestException as e:
    print(f"[{time.strftime('%H:%M:%S')}] Gangguan Jaringan Luar: {e}")

# Melakukan pengecekan kesehatan server setiap 30 detik sekali
time.sleep(30)

if __name__ == "__main__":
    monitor_site()
```

Catatan Keamanan: Sangat direkomendasikan untuk menggunakan SSH Key (Private Key) dibandingkan hardcoded password untuk lingkungan produksi demi menjaga keamanan kredensial server Anda.

Langkah 3: Pembuatan Alur Kerja Custom (Workflow)

Langkah ini menyatukan skill dan skrip di atas agar dapat dipanggil secara interaktif menggunakan pintasan perintah (shortcut) di dalam prompt.

Buatlah berkas baru bernama `start-watcher.md` di dalam direktori `.agents/workflows/`, lalu isi dengan konfigurasi makro berikut:

```
# File Workflow: Jalankan AI Watcher Otomatis
Apabila pengguna mengetikkan perintah shortcut `/watch-server`, aktifkan komponen `vercel-timeout-watcher` skill, pastikan library eksternal `paramiko` dan `requests` sudah terinstal di lingkungan Python, kemudian langsung eksekusi perintah `python watcher.py` di background terminal lokal proyek.
```

Langkah 4: Eksekusi dan Pengujian Akhir (Finish)

Setelah seluruh komponen berkas di atas tersimpan dengan baik di ruang kerja (workspace) Anda, sistem siap dijalankan:

1. Buka panel chat atau konsol agen **Antigravity** Anda.
2. Ketik perintah ringkas berikut pada kolom chat:

```
/watch-server
```

3. Agen pintar akan secara otomatis memindai struktur proyek Anda, menginstal dependensi pustaka Python yang dibutuhkan, dan langsung menghidupkan skrip pemantau di latar belakang.

Kini arsitektur Anda telah terlindungi secara mandiri. Ketika server kelebihan beban hingga mengakibatkan galat pembekuan respon di sisi Vercel, agen otomatis akan mengambil alih penanganan dan memulihkannya dalam hitungan detik tanpa membutuhkan intervensi manual dari Anda.